

1. Objetivo

Treinar e avaliar formalmente **dois modelos supervisionados** que sustentam o JobMatch AI:

Tarefa	Tipo	Alvo	Modelo escolhido
Fit x No Fit	Classificação binária	fit (0/1)	Regressão Logística
Salário da vaga	Regressão	job_salary (R\$)	Gradient Boosting Regressor

A seleção do modelo final foi feita por **validação cruzada (5-fold)**, comparando 3 algoritmos candidatos em cada tarefa.

2. Dataset

- **3.000 pares (candidato x vaga)** sintéticos, realistas e reproduzíveis (seed fixa).
- Gerado a partir de 6 arquétipos de vaga, 21 competências com peso de mercado, 3 níveis de senioridade e 3 modalidades de trabalho.
- Split **treino/teste = 80/20** (estratificado na classificação).
- Distribuição de classes: No Fit = 1.639 · Fit = 1.361.

Features de Classificação: match_ratio , n_matched , n_missing , exp_gap , salary_gap_ratio , workplace_match , n_extra_skills

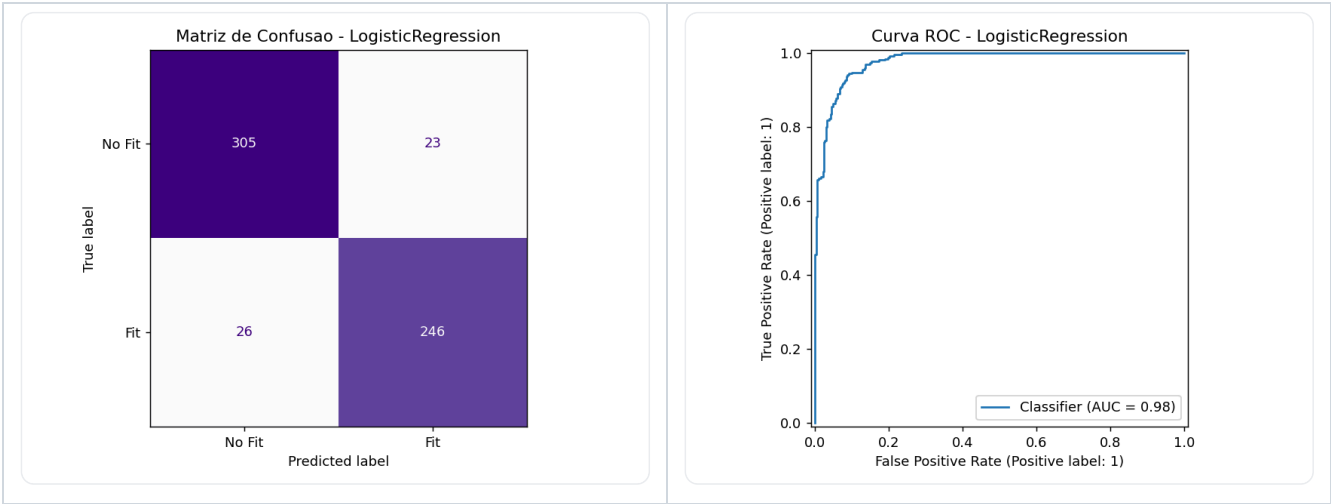
Features de Regressão: seniority_level , required_exp , n_required , avg_skill_demand , loc_factor

3. Resultados — Classificação (Fit x No Fit)

Modelo escolhido: **Regressão Logística** (maior F1 em validação cruzada).

Métrica	Valor
Acurácia	0,918
Precisão	0,915
Recall	0,904
F1-Score	0,909
ROC-AUC	0,978

Comparação de modelos (F1 em CV 5-fold): Regressão Logística **0,906** · Gradient Boosting 0,899 · Random Forest 0,886



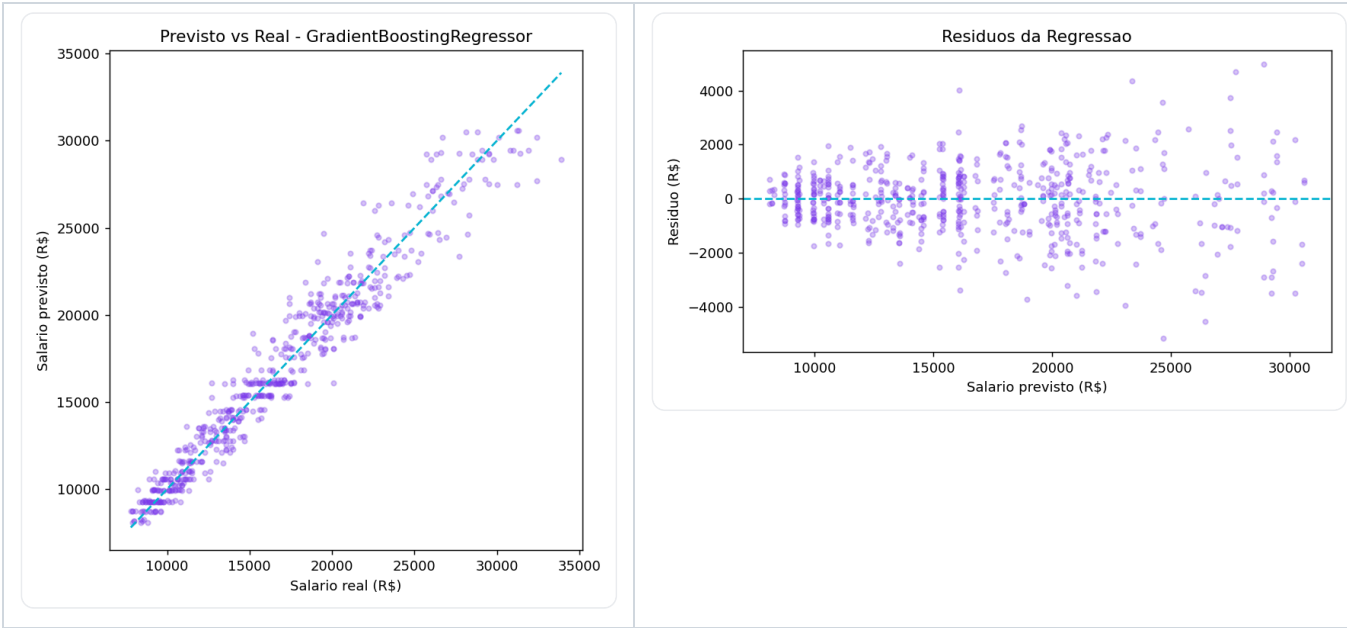
Métrica principal: F1-Score — equilibra precisão e recall, importante pelo custo assimétrico (um falso positivo faz a empresa perder tempo; um falso negativo descarta um bom candidato).

4. Resultados — Regressão (Salário)

Modelo escolhido: **Gradient Boosting Regressor** (maior R^2 em validação cruzada).

Métrica	Valor
MAE	R\$ 969,88
RMSE	R\$ 1.284,10
R^2	0,949
MAPE	5,86%

Comparação de modelos (R^2 em CV 5-fold): Gradient Boosting **0,949** · Random Forest 0,945 · Regressão Linear 0,915



Métricas principais: MAE e RMSE (erro direto em reais) e R^2 (variância explicada). O MAPE de 5,86% indica erro percentual baixo.

5. Arquitetura de produção (MLOps)

Os modelos treinados são servidos por um **microserviço Python (FastAPI)** no **Google Cloud Run**, consumido pelo backend da plataforma:

```
Frontend (Next.js) → Backend (Hono/Bun) → ML Service (FastAPI + scikit-learn)
                               /api/v1/ml/...           /predict/fit  · /predict/salary
```

- Modelos persistidos em `.joblib` e carregados na inicialização do serviço.
- Endpoints: `POST /predict/fit` (classificação) e `POST /predict/salary` (regressão).
- Tudo reproduzível com `python generate_dataset.py && python train.py (seed=42)`.